

Roteiro de laboratório — Subconsultas em T-SQL com o banco fiscal

Curso: Banco de Dados Relacional aplicado à Fiscalização Tributária

Módulo: Subconsultas em T-SQL

SGBD: Microsoft SQL Server (T-SQL)

Banco de exemplo: [curso_integridade_fiscal](#)

Base de referência: [subconsultas.md](#) e os scripts da pasta [modulo_03/banco_fiscal](#)

Como usar este roteiro

Este material foi pensado como um roteiro de laboratório, não como leitura passiva. Cada etapa tem:

- **O que fazer** → a consulta a executar
- **O que observar** → o ponto que a consulta demonstra
- **A armadilha** → o erro ou o detalhe que costuma confundir o aluno

Execute as consultas no SSMS, compare o resultado e só depois avance.

Preparação

```
USE curso_integridade_fiscal;  
GO
```

Antes de iniciar, certifique-se de que os scripts da pasta [modulo_03/banco_fiscal](#) já foram executados.

```
SELECT COUNT(*) AS qtd_nfe  
FROM dbo.nfe;  
  
SELECT COUNT(*) AS qtd_contribuintes  
FROM dbo.contribuinte;
```

Etapa 1 — Subconsulta escalar

1.1 Média de notas por contribuinte

O que fazer:

```
SELECT
  c.id_contribuinte,
  c.razao_social,
  (
    SELECT AVG(n.valor_total)
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
  ) AS media_valor_nfe
FROM dbo.contribuinte AS c
WHERE c.id_contribuinte <= 10
ORDER BY c.id_contribuinte;
```

O que observar: a subconsulta interna calcula um valor para cada linha da consulta externa. Esse é um caso clássico de subconsulta escalar.

A armadilha: se a subconsulta retornar mais de uma linha, o SQL Server gera erro. Por isso, em comparações simples com `=` é importante garantir a unicidade com `TOP (1)`, agregação ou filtro restritivo.

1.2 Última NF-e emitida por contribuinte

```
SELECT
  c.id_contribuinte,
  c.razao_social,
  (
    SELECT TOP (1) n.chave_acesso
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
    ORDER BY n.data_emissao DESC, n.id_nfe DESC
  ) AS ultima_chave
FROM dbo.contribuinte AS c
WHERE c.id_contribuinte <= 10;
```

O que observar: a subconsulta depende do valor de `c.id_contribuinte` e, por isso, é correlacionada.

Etapa 2 — IN e NOT IN

2.1 Contribuintes que emitiram alguma NF-e cancelada

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE c.id_contribuinte IN (
    SELECT n.id_emitente
    FROM dbo.nfe AS n
    WHERE n.situacao = 'CANCELADA'
)
ORDER BY c.razao_social;
```

O que observar: a subconsulta devolve um conjunto de valores que a consulta externa testa com **IN**.

2.2 Contribuintes que nunca emitiram NF-e cancelada

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE c.id_contribuinte NOT IN (
    SELECT n.id_emitente
    FROM dbo.nfe AS n
    WHERE n.situacao = 'CANCELADA'
)
ORDER BY c.razao_social;
```

A armadilha: esta forma é simples, mas pode dar resultado inesperado quando a subconsulta contém **NULL**.

Etapa 3 — EXISTS e NOT EXISTS

3.1 Emissão de nota cancelada — versão com EXISTS

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
        AND n.situacao = 'CANCELADA'
)
ORDER BY c.razao_social;
```

O que observar: **EXISTS** testa apenas se existe pelo menos uma linha. O resultado é equivalente ao **IN**, mas a semântica é mais explícita para a ideia de “existe relação”.

3.2 Contribuintes sem nenhuma NF-e cancelada

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
        AND n.situacao = 'CANCELADA'
)
ORDER BY c.razao_social;
```

A armadilha: esta é a forma recomendada para expressar “não existe” em produção, pois evita o problema do **NOT IN** com **NULL**.

Etapa 4 — A armadilha do NOT IN com NULL

O banco de exemplo tem **dbo.nfe.id_destinatario** com valores nulos em muitas notas. Essa é uma oportunidade excelente para demonstrar a armadilha.

4.1 Demonstração do problema

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE c.id_contribuinte NOT IN (
    SELECT n.id_destinatario
    FROM dbo.nfe AS n
);
```

O que observar: se a subconsulta retornar pelo menos um **NULL**, a consulta pode ficar vazia mesmo quando existirem contribuintes obviamente “ausentes” da lista.

4.2 Versão segura com **NOT EXISTS**

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    WHERE n.id_destinatario = c.id_contribuinte
);
```

Conclusão: em cenário de produção, prefira **NOT EXISTS** quando houver qualquer chance de **NULL** na coluna relacionada.

Etapa 5 — Subconsulta em **HAVING**

5.1 Regimes com média acima da média geral

```
SELECT
    c.regime_tributario,
    AVG(n.valor_total) AS media_regime
FROM dbo.contribuinte AS c
JOIN dbo.nfe AS n
    ON n.id_emitente = c.id_contribuinte
GROUP BY c.regime_tributario
HAVING AVG(n.valor_total) > (
    SELECT AVG(valor_total)
```

```
FROM dbo.nfe  
);
```

O que observar: o **HAVING** filtra grupos agregados. A subconsulta calcula uma base comparativa geral para o conjunto inteiro.

Etapa 6 — Subconsulta na cláusula **FROM** (table derived)

6.1 Resumo por regime tributário

```
SELECT  
    resumo.regime_tributario,  
    resumo.qtd_contribuintes,  
    resumo.media_nota  
FROM (  
    SELECT  
        c.regime_tributario,  
        COUNT(DISTINCT c.id_contribuinte) AS qtd_contribuintes,  
        AVG(n.valor_total) AS media_nota  
    FROM dbo.contribuinte AS c  
    JOIN dbo.nfe AS n  
        ON n.id_emitente = c.id_contribuinte  
    GROUP BY c.regime_tributario  
    ) AS resumo  
WHERE resumo.qtd_contribuintes > 5  
ORDER BY resumo.media_nota DESC;
```

O que observar: a subconsulta vira uma tabela derivada e pode ser filtrada ou reagrupada na consulta externa.

A armadilha: em T-SQL, a tabela derivada precisa de alias obrigatório.

Etapa 7 — **APPLY** como junção com subconsulta correlacionada

7.1 As 3 maiores NF-e de cada contribuinte

```
SELECT
    c.id_contribuinte,
    c.razao_social,
    top3.numero,
    top3.valor_total
FROM dbo.contribuinte AS c
CROSS APPLY (
    SELECT TOP (3)
        n.numero,
        n.valor_total
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
    ORDER BY n.valor_total DESC
) AS top3
WHERE c.id_contribuinte <= 15
ORDER BY c.id_contribuinte, top3.valor_total DESC;
```

O que observar: **CROSS APPLY** permite trazer múltiplas linhas para cada contribuinte, algo que uma subconsulta escalar tradicional não consegue fazer.

A armadilha: se a subconsulta não retornar linhas, **CROSS APPLY** elimina a linha externa; **OUTER APPLY** preserva a linha externa com **NULL**.

Etapa 8 — Exercícios práticos

1. Liste os contribuintes que emitiram notas com **situacao = 'AUTORIZADA'** usando **IN**.
 2. Reescreva o exercício 1 usando **EXISTS**.
 3. Liste contribuintes que nunca emitiram notas canceladas com **NOT EXISTS**.
 4. Mostre, com subconsulta correlacionada, a última NF-e de cada contribuinte.
 5. Crie uma consulta com **HAVING** para comparar a média de valor por regime com a média geral.
 6. Reescreva a consulta do exercício 4 usando **CROSS APPLY** para trazer as 2 notas de maior valor de cada contribuinte.
 7. Faça uma demonstração do problema de **NOT IN** com **NULL** usando **dbo.nfe.id_destinatario**.
-

Etapa 9 — Regra prática para a sala de aula

- Use **EXISTS/NOT EXISTS** quando a pergunta for “existe relação?”.
- Use **JOIN** quando for necessário trazer colunas da tabela relacionada.
- Use **CTE (WITH)** quando a consulta precisar reutilizar um resultado intermediário várias vezes.
- Evite **NOT IN** quando houver chance de **NULL** na coluna da subconsulta.
- Prefira **APPLY** quando a subconsulta correlacionada precisar retornar mais de uma linha por linha externa.